

Fresh Cluster Deployment — Step-by-Step Runbook

RKE2 on Proxmox | From Zero to Production-Ready

DevOps Team

2026-05-13

Contents

How to Use This Runbook	3
Phase 0 — Pre-Flight Checklist	4
0.1 Workstation Tools	4
0.2 SSH Key for VM Access	4
0.3 SSH Key for Proxmox Root Access	4
0.4 IP Address Planning	5
0.5 Secrets to Prepare	5
0.6 Clone the Repo	5
Phase 1 — Proxmox Preparation (Manual)	6
1.1 Create the Terraform API Token	6
1.2 Create the Ubuntu 22.04 Cloud-Init Template	6
Phase 1 — Go / No-Go	7
Phase 2 — Configure and Apply Terraform	8
2.1 Copy and Fill Proxmox tfvars	8
2.2 Copy and Fill Observability tfvars	8
2.3 Export All Secrets	9
2.4 Run Terraform (Proxmox)	9
2.5 Verify VMs are Up	10
Phase 2 — Go / No-Go	10
Phase 3 — RKE2 Cluster Installation	11
3.1 Make Scripts Executable	11
3.2 Install Masters	11
3.3 Install Workers	11
3.4 Verify the Cluster	11
Phase 3 — Go / No-Go	12
Phase 4 — Security Hardening	13
4.1 Apply Namespaces and RBAC	13
4.2 Apply Network Policies	13
4.3 Install cert-manager	13
4.4 Install External Secrets Operator	14
4.5 Generate Role Kubeconfigs	14

Phase 4 — Go / No-Go	14
Phase 5 — Observability Stack	15
5.1 Deploy Prometheus (in-cluster)	15
5.2 Verify Prometheus	15
5.3 Verify Alloy is Sending Telemetry	16
5.4 Verify in Central Grafana	16
Phase 5 — Go / No-Go	16
Phase 6 — Cilium IngressController Verification	17
6.1 Verify IngressClass and Service	17
6.2 Smoke Test — Deploy a Test App	17
Phase 6 — Go / No-Go	18
Full Automated Deployment (Alternative)	19
Partial Re-runs	19
Watching the Log	19
Post-Deployment Checklist	20
Cluster Health	20
Security	20
Observability	20
Distribute Kubeconfigs to Team	21
Troubleshooting Common Failures	22
Terraform apply fails: “connection refused” or “timeout”	22
VM created but SSH times out	22
RKE2 install fails on master-1	22
Masters join but stay NotReady	22
Prometheus scrape targets are down	23
Alloy not sending logs/metrics	23
cert-manager ClusterIssuer stuck “not ready”	23
Quick Reference Card	24
Essential Commands	24
Key File Locations	24
Files That Must NEVER Be Committed to Git	24
Deployment Timeline (Expected)	25

How to Use This Runbook

This document walks you through deploying a brand-new RKE2 Kubernetes cluster on Proxmox from scratch. Follow the phases in order. Every phase ends with a **Go / No-Go gate** — do not move to the next phase until all checks pass.

Total time: ~35-50 minutes unattended after Phase 1 is complete.

Deployment model:

- Phase 1 is manual (one-time Proxmox prep)
 - Phases 2-6 run via a single script: `./scripts/deploy.sh`
 - You can also run each phase individually if needed
-

Phase 0 — Pre-Flight Checklist

Complete this before touching anything. A missing item will cause a phase to fail mid-run.

0.1 Workstation Tools

Install on the machine you run deployments from:

```
# Verify versions
terraform version      # need >= 1.6.0
kubectl version --client # need >= 1.29
helm version           # need >= 3.14
openssl version
jq --version
ssh -V
```

Install missing tools:

```
# Terraform
wget https://releases.hashicorp.com/terraform/1.8.5/terraform_1.8.5_linux_amd64.zip
unzip terraform_1.8.5_linux_amd64.zip && sudo mv terraform /usr/local/bin/

# kubectl
curl -LO "https://dl.k8s.io/release/$(curl -Ls https://dl.k8s.io/release/stable.txt)/"
chmod +x kubectl && sudo mv kubectl /usr/local/bin/

# helm
curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash
```

0.2 SSH Key for VM Access

```
# Generate a dedicated key for cluster VM access
ssh-keygen -t ed25519 -f ~/.ssh/rke2_cluster_id -C "rke2-cluster-deploy" -N ""
chmod 600 ~/.ssh/rke2_cluster_id

# Copy public key – you will paste this into terraform.tfvars
cat ~/.ssh/rke2_cluster_id.pub
```

0.3 SSH Key for Proxmox Root Access

Terraform needs root SSH access to Proxmox for file uploads (VM configs):

```
# Generate or reuse an existing root SSH key
ssh-keygen -t ed25519 -f ~/.ssh/proxmox_id_rsa -C "proxmox-root" -N ""
chmod 600 ~/.ssh/proxmox_id_rsa

# Copy public key to Proxmox host (run this once)
ssh-copy-id -i ~/.ssh/proxmox_id_rsa.pub root@YOUR_PROXMOX_IP
```

```
# Verify it works
ssh -i ~/.ssh/proxmox_id_rsa root@YOUR_PROXMOX_IP "hostname"
```

0.4 IP Address Planning

Decide and reserve these IPs in your network **before starting**. None of these should be assigned to any existing device.

Purpose	Variable	Example
Master node 1	master_ip_addresses[0]	192.168.10.101
Master node 2	master_ip_addresses[1]	192.168.10.102
Master node 3	master_ip_addresses[2]	192.168.10.103
Worker node 1	worker_ip_addresses[0]	192.168.10.111
Worker node 2	worker_ip_addresses[1]	192.168.10.112
Control plane VIP	control_plane_vip	192.168.10.100
Ingress VIP	(set in Cilium config)	192.168.10.200

The VIPs must be **free IPs** — not allocated by DHCP and not assigned to any host. Reserve them in your router/DHCP server as exclusions before continuing.

0.5 Secrets to Prepare

Gather these values now — you will export them as environment variables before deploying:

Value	Where to Get It
Proxmox API password	Proxmox terraform@pve user password
Central Mimir push URL	Your Mimir admin (e.g. https://mimir.yourdomain.com/api/v1/push)
Central Loki push URL	Your Loki admin (e.g. https://loki.yourdomain.com)
Mimir basic-auth password	Your Mimir admin (empty if no auth)
Loki basic-auth password	Your Loki admin (empty if no auth)
Slack webhook URL	Slack App settings → Incoming Webhooks
PagerDuty routing key	PagerDuty → Services → Integration Key

0.6 Clone the Repo

```
git clone <your-infra-repo-url> infra-setup
cd infra-setup
```

```
# Confirm .secrets/ is gitignored
grep -q ".secrets" .gitignore && echo "OK" || echo "ADD .secrets TO .gitignore NOW"
```

Phase 1 — Proxmox Preparation (Manual)

This phase is one-time manual work. Once done, never needs repeating.

1.1 Create the Terraform API Token

Run these commands on the Proxmox host as root:

```
ssh root@YOUR_PROXMUX_IP

# Create user and role
pveum user add terraform@pve
pveum role add TerraformRole -privs \
  "VM.Allocate VM.Clone VM.Config.CDR0M VM.Config.CPU \
  VM.Config.Cloudinit VM.Config.Disk VM.Config.HWType \
  VM.Config.Memory VM.Config.Network VM.Config.Options \
  VM.Monitor VM.Audit VM.PowerMgmt \
  Datastore.AllocateSpace Datastore.Audit SDN.Use"

pveum aclmod / -user terraform@pve -role TerraformRole

# Create API token (no expiry – rotate manually if compromised)
pveum user token add terraform@pve terraform --expire 0
# OUTPUT: Secret = <TOKEN VALUE> ← save this as your TF_VAR_proxmox_password
```

Important: The token secret is shown only once. Copy it immediately.

1.2 Create the Ubuntu 22.04 Cloud-Init Template

Still on the Proxmox host as root:

```
TEMPLATE_ID=9000

# Download Ubuntu 22.04 cloud image
wget -q https://cloud-images.ubuntu.com/jammy/current/jammy-server-cloudimg-amd64.img
-O /tmp/ubuntu-2204.img

# Create template VM
qm create $TEMPLATE_ID --memory 2048 --cores 2 \
  --name ubuntu-2204-template \
  --net0 virtio,bridge=vmbr0

# Import disk
qm importdisk $TEMPLATE_ID /tmp/ubuntu-2204.img local-lvm

# Configure boot order and cloud-init
qm set $TEMPLATE_ID \
  --scsihw virtio-scsi-pci \
  --scsi0 local-lvm:vm-${TEMPLATE_ID}-disk-0,discard=on \
  --ide2 local-lvm:cloudinit \
  --boot c \
```

```
--bootdisk scsi0 \  
--serial0 socket \  
--vga serial0 \  
--agent enabled=1 \  
--ostype l26  
  
# Convert to template  
qm template $TEMPLATE_ID  
  
# Verify  
qm list | grep $TEMPLATE_ID  
# Expected: shows VM 9000 as a template
```

Phase 1 — Go / No-Go

- ☐ terraform@pve token created and saved
 - ☐ VM 9000 (or your chosen ID) shows as template in `qm list`
 - ☐ SSH key (proxmox_id_rsa) works: `ssh -i ~/.ssh/proxmox_id_rsa root@PROXMOX_IP hostname`
 - ☐ All node IPs planned and reserved in your network
-

Phase 2 — Configure and Apply Terraform

2.1 Copy and Fill Proxmox tfvars

```
cd terraform/proxmox
cp terraform.tfvars.example terraform.tfvars
```

Open terraform.tfvars and fill in **every** value. Key fields:

```
# Connection
proxmox_api_url      = "https://192.168.1.10:8006/api2/json"
proxmox_username     = "terraform@pve"
proxmox_tls_insecure = false           # true only if Proxmox uses self-signed
proxmox_ssh_user     = "root"
proxmox_ssh_private_key_path = "~/.ssh/proxmox_id_rsa"
proxmox_node         = "pve"           # run: pvesh get /nodes | grep node

# Network
network_bridge      = "vmbr0"
network_gateway     = "192.168.10.1"
dns_servers         = ["192.168.10.1", "8.8.8.8"]
control_plane_vip   = "192.168.10.100" # must be a FREE IP
control_plane_vip_interface = "eth0"

# Template
vm_template_id = 9000 # must match what you created in Phase

# SSH
vm_ssh_public_key      = "ssh-ed25519 AAAA..." # paste output of: cat ~/.ssh/rke2_c
vm_ssh_private_key_path = "~/.ssh/rke2_cluster_id"
vm_user                = "ubuntu"

# Node IPs – must match your IP plan
master_ip_addresses = ["192.168.10.101", "192.168.10.102", "192.168.10.103"]
worker_ip_addresses = ["192.168.10.111", "192.168.10.112"]

# RKE2
rke2_version = "v1.29.4+rke2r1"
rke2_cni     = "cilium"
cluster_name = "rke2-prod"
environment  = "production"
```

2.2 Copy and Fill Observability tfvars

```
cd ../observability
cp terraform.tfvars.example terraform.tfvars
```

Open terraform.tfvars and fill in the centralized endpoints:


```

cluster_name = "rke2-prod"
environment  = "production"

# YOUR centralized Mimir (required)
central_mimir_url      = "https://mimir.yourdomain.com/api/v1/push"
central_mimir_username = ""      # leave empty if no auth

# YOUR centralized Loki (required)
central_loki_url        = "https://loki.yourdomain.com"
central_loki_username   = ""      # leave empty if no auth

loki_tenant_id = "rke2-prod"    # appears as X-Scope-OrgID header to Loki

# Prometheus sizing (adjust to your hardware)
prometheus_replicas      = 2
prometheus_storage_size  = "20Gi"
prometheus_retention_days = 3

# Alertmanager
alertmanager_email_to    = "devops@yourcompany.com"
alertmanager_smtp_host   = "smtp.yourcompany.com"

```

Also set in rke2/configs/rke2-cilium-config.yaml:

```

ingressController:
  service:
    loadBalancerIP: "192.168.10.200"    # YOUR ingress VIP – must be a free IP

```

2.3 Export All Secrets

```

# Required
export TF_VAR_proxmox_password="your-proxmox-api-token-secret"

# Mimir/Loki auth (leave empty string if no auth on your central stack)
export TF_VAR_central_mimir_password=""
export TF_VAR_central_loki_password=""

# Alerting (at least one)
export TF_VAR_alertmanager_slack_webhook="https://hooks.slack.com/services/..."
export TF_VAR_alertmanager_pagerduty_key="your-pd-routing-key"

```

Tip: Put these exports in a file like `.secrets/env.sh`, `chmod 600` it, and source `.secrets/env.sh` before each deploy session. Never commit this file.

2.4 Run Terraform (Proxmox)

```

cd terraform/proxmox
terraform init

```

```

terraform validate          # must show: Success!

# Review what will be created
terraform plan -out=cluster.tfplan
# Expected: ~25-30 resources to add (VMs, disks, null_resources)

# Apply
terraform apply cluster.tfplan
# Expected time: 5-10 minutes

```

What Terraform does:

1. Creates 3 master VMs cloned from template
2. Creates 2 worker VMs cloned from template
3. Formats etcd disk (/dev/vdb) on each master as ext4 → mounts at /var/lib/rancher/rke2/serve
4. Formats data disk (/dev/vdb) on each worker as XFS → mounts at /var/lib/rancher
5. Applies kernel tuning (vm.swappiness=0, inotify, bridge netfilter) to all nodes
6. Generates rke2/configs/inventory.ini with all node IPs
7. Generates per-node RKE2 config YAMLS in rke2/configs/

2.5 Verify VMs are Up

```

# All VMs should be reachable via SSH
for ip in 192.168.10.101 192.168.10.102 192.168.10.103 192.168.10.111 192.168.10.112;
do
  ssh -i ~/.ssh/rke2_cluster_id -o StrictHostKeyChecking=no ubuntu@$ip \
    "echo $ip OK" 2>/dev/null || echo "$ip NOT READY"
done

```

All should print <IP> OK.

Phase 2 — Go / No-Go

- ☐ terraform apply completed with 0 errors
 - ☐ All 5 VMs are RUNNING in Proxmox UI
 - ☐ All 5 VMs respond to SSH
 - ☐ rke2/configs/inventory.ini exists and lists all nodes
-

Phase 3 — RKE2 Cluster Installation

This phase installs RKE2 on all nodes, deploys kube-vip for HA, configures Cilium CNI, and installs Grafana Alloy as a systemd service on every VM.

3.1 Make Scripts Executable

```
cd <repo-root>
chmod +x scripts/deploy.sh rke2/scripts/*.sh rbac/scripts/*.sh
```

3.2 Install Masters

```
./rke2/scripts/install-master.sh
```

What it does (in order):

1. Connects to master-1 (init node), uploads Cilium HelmChartConfig and RKE2 config
2. Starts RKE2 server on master-1 → bootstraps etcd
3. Saves cluster join token to .secrets/rke2-cluster-token
4. Waits for master-1 API to be healthy
5. Deploys kube-vip DaemonSet → VIP 192.168.10.100 becomes active
6. Saves admin kubeconfig to .secrets/kubeconfig-admin.yaml
7. Connects to master-2 and master-3, starts RKE2 configured to join via VIP
8. Installs Grafana Alloy (systemd) on all masters with role=master

Expected output (master-1 bootstrap):

```
[INFO] Installing RKE2 on rke2-master-1 (init node)...
[INFO] Uploading Cilium config...
[INFO] Starting RKE2 server...
[INFO] Waiting for API server at https://192.168.10.100:6443...
[OK]   API server ready
[INFO] Deploying kube-vip...
[OK]   VIP 192.168.10.100 is active
[INFO] Installing Alloy on rke2-master-1...
[OK]   Alloy active on rke2-master-1
```

3.3 Install Workers

```
./rke2/scripts/install-worker.sh
```

Workers join the cluster via the VIP (192.168.10.100:9345) using the token from .secrets/rke2-cluster-token. Alloy is installed on each worker with role=worker (no etcd scrape block).

3.4 Verify the Cluster

```
export KUBECONFIG=.secrets/kubeconfig-admin.yaml
```

All 5 nodes should be Ready

```
kubectl get nodes -o wide
```

Expected output:

NAME	STATUS	ROLES	AGE	VERSION
rke2-master-1	Ready	control-plane,etcd,master	8m	v1.29.4+rke2r1
rke2-master-2	Ready	control-plane,etcd,master	5m	v1.29.4+rke2r1
rke2-master-3	Ready	control-plane,etcd,master	3m	v1.29.4+rke2r1
rke2-worker-1	Ready	<none>	2m	v1.29.4+rke2r1
rke2-worker-2	Ready	<none>	1m	v1.29.4+rke2r1

Cilium CNI is running

```
kubectl get pods -n kube-system -l app.kubernetes.io/name=cilium
```

Cilium IngressClass is registered

```
kubectl get ingressclass cilium
```

Alloy is running on each node as systemd

```
ssh ubuntu@192.168.10.101 "systemctl is-active alloy" # → active
```

```
ssh ubuntu@192.168.10.111 "systemctl is-active alloy" # → active
```

Phase 3 — Go / No-Go

- ☐ All 5 nodes show STATUS=Ready
 - ☐ Zero nodes show NotReady or Unknown
 - ☐ `kubectl get pods -n kube-system` — all pods Running (give Cilium 2-3 minutes)
 - ☐ `kubectl get ingressclass cilium` returns a result
 - ☐ `systemctl is-active alloy` returns active on at least one master and one worker
 - ☐ `.secrets/kubeconfig-admin.yaml` exists
-

Phase 4 — Security Hardening

4.1 Apply Namespaces and RBAC

```
export KUBECONFIG=.secrets/kubeconfig-admin.yaml

# Namespaces with Pod Security Standards labels
kubectl apply -f rbac/00-namespace-setup.yaml

# Team RBAC roles
kubectl apply -f rbac/01-senior-devops.yaml
kubectl apply -f rbac/02-junior-devops.yaml
kubectl apply -f rbac/03-developer.yaml
kubectl apply -f rbac/04-read-only-auditor.yaml

# Verify
kubectl get namespaces
# Should list: production, staging, development, monitoring, cert-manager
kubectl get clusterrolebindings | grep devops
```

4.2 Apply Network Policies

```
kubectl apply -f security/network-policies/

# Verify default-deny is in place
kubectl get networkpolicies -n production
kubectl get networkpolicies -n staging
kubectl get networkpolicies -n monitoring
```

4.3 Install cert-manager

```
helm repo add jetstack https://charts.jetstack.io --force-update
helm repo update

helm install cert-manager jetstack/cert-manager \
  --namespace cert-manager \
  --create-namespace \
  --set installCRDs=true \
  --version v1.14.5 \
  --wait

# Edit your email address before applying
# security/tls/cluster-issuer.yaml → spec.acme.email: your@email.com
kubectl apply -f security/tls/cluster-issuer.yaml

# Verify issuers
kubectl get clusterissuers
# Expected: letsencrypt-prod, letsencrypt-staging, cluster-ca-issuer — all READY=True
```

4.4 Install External Secrets Operator

```
helm repo add external-secrets https://charts.external-secrets.io --force-update
helm repo update
```

```
helm install external-secrets external-secrets/external-secrets \
  --namespace external-secrets \
  --create-namespace \
  --wait
```

```
# Edit your Vault URL before applying:
# security/secrets/external-secrets-operator.yaml → spec.provider.vault.server
kubectl apply -f security/secrets/external-secrets-operator.yaml
```

```
# Verify
kubectl get clustersecretstore
```

4.5 Generate Role Kubeconfigs

```
./rbac/scripts/generate-kubeconfigs.sh
# Output: rbac/kubeconfigs/
ls -la rbac/kubeconfigs/
```

Expected files:

```
kubeconfig-senior-devops-user.yaml    (valid 365 days)
kubeconfig-junior-devops-user.yaml    (valid 180 days)
kubeconfig-developer-user.yaml        (valid 180 days)
kubeconfig-auditor-user.yaml          (valid 90 days)
```

Distribute these to your team via an encrypted channel — never email or Slack DM.

Phase 4 — Go / No-Go

- ☐ Namespaces production, staging, development, monitoring, cert-manager exist
 - ☐ `kubectl get clusterrolebindings | grep devops` shows entries
 - ☐ `kubectl get networkpolicies -n production` shows deny-all policy
 - ☐ `kubectl get pods -n cert-manager` — all Running
 - ☐ `kubectl get clusterissuers` — all show READY=True
 - ☐ `kubectl get pods -n external-secrets` — all Running
 - ☐ 4 kubeconfig files exist in `rbac/kubeconfigs/`
-

Phase 5 — Observability Stack

5.1 Deploy Prometheus (in-cluster)

```
cd terraform/observability

# Ensure secrets are still exported in your shell
echo $TF_VAR_proxmox_password      # should not be empty
echo $TF_VAR_central_mimir_password
echo $TF_VAR_alertmanager_slack_webhook

helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update

terraform init
terraform validate

terraform plan -out=obs.tfplan
terraform apply -auto-approve obs.tfplan
```

What is deployed:

Component	Where
Prometheus x2 (HA)	monitoring namespace — remote-writes to central Mimir
Alertmanager x2	monitoring namespace — routes to Slack/PagerDuty/email
kube-state-metrics	monitoring namespace — Kubernetes object metrics
node-exporter	NOT deployed — Alloy on each VM replaces it
Grafana	NOT deployed — use your central Grafana

5.2 Verify Prometheus

```
export KUBECONFIG=.secrets/kubeconfig-admin.yaml

kubectl get pods -n monitoring
# All pods should be Running

# Port-forward and open in browser
kubectl port-forward -n monitoring svc/kube-prometheus-stack-prometheus 9090:9090 &

# Check all scrape targets are UP
curl -s http://localhost:9090/api/v1/targets | jq '.data.activeTargets | map(select(.))'
# Expected: 0 (all targets healthy)
```

```
# Check remote-write is configured
curl -s http://localhost:9090/api/v1/query?query=up | jq '.status'
# Expected: "success"
```

5.3 Verify Alloy is Sending Telemetry

```
# On a master node – check Alloy health and recent log output
ssh ubuntu@192.168.10.101 "journalctl -u alloy -n 30 --no-pager 2>&1 | grep -iE 'error'"

# On a worker node
ssh ubuntu@192.168.10.111 "journalctl -u alloy -n 30 --no-pager 2>&1 | grep -iE 'error'"
```

Look for lines showing successful exports (no error lines). Normal output includes:

```
component=prometheus.remote_write ... exported=12345
component=loki.write ... sent_bytes=...
```

5.4 Verify in Central Grafana

In your central Grafana:

1. **Metrics (Mimir datasource):** Go to Explore → select Mimir → run `up{cluster="rke2-prod"}` → should return results for all nodes
2. **Logs (Loki datasource):** Go to Explore → select Loki → query `{cluster="rke2-prod"}` → should show pod logs and systemd logs

Recommended dashboard imports (filter all by `cluster="rke2-prod"`):

Dashboard	Grafana ID	Datasource
Kubernetes Cluster Overview	7249	Mimir
Node Exporter Full (Alloy)	1860	Mimir
etcd	3070	Mimir
Loki Logs Explorer	13639	Loki
Cilium / Hubble	16611	Mimir

Phase 5 — Go / No-Go

- ☐ All pods in monitoring namespace are Running
- ☐ Prometheus scrape targets: 0 targets in down state
- ☐ `up{cluster="rke2-prod"}` returns data in central Grafana (Mimir)
- ☐ `{cluster="rke2-prod"}` returns logs in central Grafana (Loki)
- ☐ No error lines in Alloy logs on any node

Phase 6 — Cilium IngressController Verification

Cilium IngressController was already deployed automatically in Phase 3 via the `rke2-cilium-config.yaml` HelmChartConfig. This phase is verification only.

6.1 Verify IngressClass and Service

```
export KUBECONFIG=.secrets/kubeconfig-admin.yaml

# IngressClass must exist
kubectl get ingressclass cilium
# Expected: cilium    cilium.io/ingress-controller    <date>

# LoadBalancer service for the ingress (kube-vip assigns the IP)
kubectl get svc -n kube-system -l app.kubernetes.io/name=cilium-ingress
# Expected: EXTERNAL-IP = 192.168.10.200 (your ingress VIP)

# Hubble observability UI
kubectl get pods -n kube-system -l app.kubernetes.io/name=hubble-ui
# Expected: Running

# Hubble relay
kubectl get pods -n kube-system -l app.kubernetes.io/name=hubble-relay
# Expected: Running
```

6.2 Smoke Test — Deploy a Test App

```
kubectl apply -f - <<EOF
apiVersion: apps/v1
kind: Deployment
metadata:
  name: smoke-test
  namespace: staging
spec:
  replicas: 1
  selector:
    matchLabels:
      app: smoke-test
  template:
    metadata:
      labels:
        app: smoke-test
    spec:
      containers:
        - name: nginx
          image: nginx:alpine
          ports:
            - containerPort: 80
<<EOF
```

```

---
apiVersion: v1
kind: Service
metadata:
  name: smoke-test
  namespace: staging
spec:
  selector:
    app: smoke-test
  ports:
    - port: 80
---
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: smoke-test
  namespace: staging
spec:
  ingressClassName: cilium
  rules:
    - host: smoke-test.cluster.internal
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: smoke-test
                port:
                  number: 80
EOF

# Wait for pod to be ready
kubectl rollout status deployment/smoke-test -n staging

# Test routing (add Host header to hit the ingress)
curl -H "Host: smoke-test.cluster.internal" http://192.168.10.200/
# Expected: nginx welcome page HTML

# Clean up
kubectl delete -n staging deployment/smoke-test service/smoke-test ingress/smoke-test

```

Phase 6 — Go / No-Go

- ☐ `kubectl get ingressclass cilium` returns a result
- ☐ Cilium ingress service has EXTERNAL-IP = 192.168.10.200
- ☐ Smoke test curl returns HTTP 200 with nginx HTML
- ☐ Hubble UI and relay pods are Running

Full Automated Deployment (Alternative)

Instead of running each phase manually, use the deploy script which runs all phases:

```
cd <repo-root>

# 1. Export all secrets first
source .secrets/env.sh    # or export them inline

# 2. Ensure tfvars are filled in
ls terraform/proxmox/terraform.tfvars    # must exist
ls terraform/observability/terraform.tfvars # must exist

# 3. Run everything
chmod +x scripts/deploy.sh
./scripts/deploy.sh
```

Partial Re-runs

```
# Resume from a phase (e.g. after fixing a Phase 4 error)
./scripts/deploy.sh --from phase4

# Run only one phase
./scripts/deploy.sh --only phase5

# Validate inputs without applying anything
./scripts/deploy.sh --dry-run
```

Watching the Log

The deploy script writes a timestamped log:

```
# In a second terminal, tail the log
tail -f /tmp/deploy-<timestamp>.log
```

Post-Deployment Checklist

Run these after all 6 phases pass their Go / No-Go gates.

Cluster Health

```
export KUBECONFIG=.secrets/kubeconfig-admin.yaml

# All nodes Ready
kubectl get nodes
# No not-ready or unknown nodes

# No unhealthy pods anywhere
kubectl get pods -A --field-selector=status.phase!=Running,status.phase!=Succeeded \
  | grep -v Completed
# Expected: empty output (or only Completed/Succeeded jobs)

# etcd cluster is healthy (run on any master)
ssh ubuntu@192.168.10.101 \
  "ETCDCTL_ENDPOINTS='https://127.0.0.1:2379' \
  ETCDCTL_CACERT='/var/lib/rancher/rke2/server/tls/etcd/server-ca.crt' \
  ETCDCTL_CERT='/var/lib/rancher/rke2/server/tls/etcd/server-client.crt' \
  ETCDCTL_KEY='/var/lib/rancher/rke2/server/tls/etcd/server-client.key' \
  ETCDCTL_API=3 /var/lib/rancher/rke2/bin/etcdctl endpoint health"
# Expected: all 3 endpoints → healthy: true
```

Security

```
# Each role kubeconfig works correctly
export KUBECONFIG=rbac/kubeconfigs/kubeconfig-senior-devops-user.yaml
kubectl get pods -n production          # should work
kubectl delete namespace production     # should be DENIED

export KUBECONFIG=rbac/kubeconfigs/kubeconfig-auditor-user.yaml
kubectl get pods -A                     # should work (read-only)
kubectl create ns test                   # should be DENIED

# Reset to admin
export KUBECONFIG=.secrets/kubeconfig-admin.yaml

# Verify no secrets are committed to git
git status --short | grep -E "\.tfvars$|kubeconfig|\.secrets"
# Expected: empty output
```

Observability

```

# Prometheus targets – 0 down
kubectl port-forward -n monitoring svc/kube-prometheus-stack-prometheus 9090:9090 &
sleep 2
curl -s http://localhost:9090/api/v1/targets | \
  jq '[.data.activeTargets[] | select(.health != "up")] | length'
# Expected: 0

# Alertmanager
kubectl port-forward -n monitoring svc/kube-prometheus-stack-alertmanager 9093:9093 &
sleep 2
curl -s http://localhost:9093/api/v2/status | jq '.cluster.status'
# Expected: "ready"

# Kill port-forwards
kill %1 %2 2>/dev/null

```

Distribute Kubeconfigs to Team

```

# Securely send to each person – options:
# 1. Password-protected ZIP
zip --password <temp-password> team-kubeconfigs.zip rbac/kubeconfigs/*.yaml

# 2. Age-encrypted (recommended)
# age -r <recipient-pubkey> -o devops.yaml.age rbac/kubeconfigs/kubeconfig-senior-dev

# NEVER distribute via Slack, email attachment, or shared folder

```

Troubleshooting Common Failures

Terraform apply fails: “connection refused” or “timeout”

```
# 1. Verify Proxmox API is reachable
curl -sk https://YOUR_PROXMOX_IP:8006/api2/json/version | jq .data.version

# 2. Verify SSH key works
ssh -i ~/.ssh/proxmox_id_rsa root@YOUR_PROXMOX_IP "hostname"

# 3. If TLS error – set proxmox_tls_insecure = true in tfvars (dev only)
```

VM created but SSH times out

```
# Check cloud-init log on the VM from Proxmox console
# Proxmox UI → VM → Console
cat /var/log/cloud-init-output.log

# Or check from Proxmox host
qm terminal 300 # opens serial console for VM 300
```

RKE2 install fails on master-1

```
# Check RKE2 server logs on master-1
ssh ubuntu@192.168.10.101 "sudo journalctl -u rke2-server -n 100 --no-pager"

# Common causes:
# - etcd disk not mounted at /var/lib/rancher/rke2/server/db
ssh ubuntu@192.168.10.101 "df -h | grep rancher"

# - VIP already in use (kube-vip conflict)
ping 192.168.10.100 # should NOT respond before kube-vip is up
```

Masters join but stay NotReady

```
# Check Cilium pods
kubectl get pods -n kube-system -l app.kubernetes.io/name=cilium

# If Cilium is crashlooping, check VIP placeholder was replaced
ssh ubuntu@192.168.10.102 "cat /etc/rancher/rke2/config.yaml | grep server"
# Expected: server: https://192.168.10.100:9345

# Check Cilium config was applied
kubectl get helmchartconfig -n kube-system rke2-cilium -o yaml | grep k8sServiceHost
# Expected: k8sServiceHost: "192.168.10.100"
```

Prometheus scrape targets are down

```
# Check ServiceMonitor labels
kubectl get servicemonitors -n monitoring -o yaml | grep release

# Common fix: Prometheus selector must match release label
kubectl get prometheuses -n monitoring -o yaml | grep serviceMonitorSelector

# Check Prometheus config is loaded
kubectl port-forward -n monitoring svc/kube-prometheus-stack-prometheus 9090:9090 &
curl http://localhost:9090/api/v1/labels
```

Alloy not sending logs/metrics

```
# On any node
ssh ubuntu@192.168.10.101

# Check Alloy status
systemctl status alloy
journalctl -u alloy -n 50 --no-pager

# Check config was rendered correctly
cat /etc/alloy/config.alloy | grep -E "url|cluster_name|tenant"

# Restart Alloy
sudo systemctl restart alloy
sleep 5
systemctl is-active alloy # should be: active
```

cert-manager ClusterIssuer stuck “not ready”

```
# Check cert-manager pods
kubectl get pods -n cert-manager

# Describe the issuer for detailed error
kubectl describe clusterissuer letsencrypt-prod

# Common causes:
# - Email not set in cluster-issuer.yaml
# - HTTP-01 challenge needs ingress to be reachable from internet
# - Use letsencrypt-staging first to rule out rate limits
```

Quick Reference Card

Essential Commands

```
# Set kubeconfig
export KUBECONFIG=.secrets/kubeconfig-admin.yaml

# Cluster health
kubectl get nodes
kubectl get pods -A | grep -v Running | grep -v Completed

# Restart a node's RKE2 (if stuck)
ssh ubuntu@<node-ip> "sudo systemctl restart rke2-server" # master
ssh ubuntu@<node-ip> "sudo systemctl restart rke2-agent"  # worker

# etcd snapshot (run on any master)
ssh ubuntu@192.168.10.101 "sudo rke2 etcd-snapshot save --name manual-$(date +%Y%m%d)"

# Scale workers – edit worker_count in tfvars, then:
cd terraform/proxmox && terraform apply
# RKE2 join happens automatically via install-worker.sh

# Rotate kubeconfigs (before expiry)
./rbac/scripts/generate-kubeconfigs.sh
```

Key File Locations

File	Purpose
.secrets/kubeconfig-admin.yaml	Admin kubectl access
.secrets/rke2-cluster-token	Node join token — protect this
.secrets/ssh-key	VM SSH private key
terraform/proxmox/terraform.tfvars	All infrastructure variables
terraform/observability/terraform.tfvars	Monitoring URLs and auth
rke2/configs/inventory.ini	Auto-generated node list
rke2/configs/alloy-config.alloy.tpl	Alloy template — edit to change telemetry
rke2/configs/rke2-cilium-config.yaml	Cilium settings — IngressController, Hubble
rbac/kubeconfigs/	Team kubeconfigs — distribute securely

Files That Must NEVER Be Committed to Git

```
.secrets/
terraform/proxmox/terraform.tfvars
terraform/observability/terraform.tfvars
*.pem *.key *.p12
rbac/kubeconfigs/
```


Verify with: `git status --short` — these paths must not appear.

Deployment Timeline (Expected)

Phase	Duration
Phase 0 — Pre-flight	~15 min (one-time)
Phase 1 — Proxmox prep	~10 min (one-time)
Phase 2 — Terraform VMs	~8-12 min
Phase 3 — RKE2 + Alloy	~12-18 min
Phase 4 — Security	~5 min
Phase 5 — Observability	~5-8 min
Phase 6 — Verification	~2 min
Total (Phases 2-6)	~35-50 min

This runbook covers a clean first-time deployment. For scaling, upgrades, and backup procedures see the companion documents in docs/.

Generated: 2026-05-13 | Cluster: rke2-prod